

LAB MANUAL – DISTRIBUTED COMPUTING

Study Guide

Prof. Mohd Affan Sheikh
Assistant Professor
MCA, PIET.
Parul University

SR. NO.	Date	Practical Title	Page no.
1		Concurrent Echo Client Server Application	
2		Swapping Variable Values Using IPC Through Thread Programming in Java	
3		Implement Remote Procedure Call (RPC) Chat Server	
4		Implement Remote Method Invocation (RMI)	
5		Increment a Counter in Shared Memory	
6		Implement Stateful Server and Stateless Server	
7		Implement Network File System (NFS)	
8		RPCgen Protocol Compiler – Minimum of Two Numbers	
9		Non-token Based Algorithm for Mutual Exclusion	
10		Edge Chasing Distributed Deadlock Detection Algorithm	

Practical No. 1

Concurrent Echo Client Server Application

AIM: To implement a concurrent TCP echo server that can handle multiple clients.

Theory: The server creates a separate thread for each connected client. The client sends a message and the server sends the same message back.

Program:

File name: Server.java

```
import java.io.*;
import java.net.*;

public class EchoServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5000);
        System.out.println("Server started on port 5000");

        while (true) {
            Socket s = ss.accept();
            new Thread(() -> handle(s)).start();
        }
    }

    static void handle(Socket s) {
        try {
            BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream()));
            PrintWriter out = new PrintWriter(s.getOutputStream(), true);

            String msg;
            while ((msg = in.readLine()) != null) {
                System.out.println("Client: " + msg);
                out.println("Echo: " + msg);
            }
            s.close();
        } catch (Exception e) {
            System.out.println("Client disconnected");
        }
    }
}
```

File name: Client.java

```
import java.io.*;
import java.net.*;

public class EchoClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5000);
        BufferedReader key = new BufferedReader(new InputStreamReader(System.in));
        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);

        System.out.print("Enter message: ");
        String msg = key.readLine();
        out.println(msg);
        System.out.println("Server: " + in.readLine());
        s.close();
    }
}
```

Commands to Execute:

javac Server.java Client.java

Terminal 1: Server.java

Terminal 2: Client.java

Output:

Server: Echo: Hello Distributed Computing

Practical No. 2

Swapping Variable Values Using IPC Through Thread Programming

AIM: To swap two variable values using a shared object accessed by threads.

Theory: Threads in the same process share memory. Here one thread performs the swap while another thread reads the updated values.

Program:

File Name: Swap.java

```
class Data {
    int a = 10, b = 20;

    synchronized void swap() {
        int temp = a;
        a = b;
        b = temp;
        System.out.println("After swap: a = " + a + ", b = " + b);
    }
}

public class SwapJava {
    public static void main(String[] args) throws Exception {
        Data d = new Data();

        Thread t1 = new Thread() -> d.swap();
        Thread t2 = new Thread() ->
            System.out.println("Read values: a = " + d.a + ", b = " + d.b));

        t1.start();
        t1.join();
        t2.start();
    }
}
```

Commands to Execute:

```
javac Swap.java  
Swap.java
```

Output (Java):

After swap: a = 20, b = 10

Practical No. 3

Remote Procedure Call (RPC) Chat Server Implementation

AIM: To implement a simple RPC-style chat between a client and server.

Theory: RPC allows a client to request an operation from a remote server. The client sends a request and receives the server's response over a socket.

Program:

File name: RPCServer.java

```
import java.io.*;
import java.net.*;

public class RPCServer {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(6000);
        System.out.println("RPC Server started");
        Socket s = ss.accept();

        BufferedReader in = new BufferedReader(
            new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(s.getOutputStream(), true);

        String msg = in.readLine();
        System.out.println("Client: " + msg);
        out.println("Server received: " + msg);
        s.close(); ss.close();
    }
}
```

File Name: RPCClient.java

```
import java.io.*;
import java.net.*;

public class RPCClient {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 6000);
        BufferedReader key = new BufferedReader(new InputStreamReader(System.in));
```

```
BufferedReader in = new BufferedReader(  
    new InputStreamReader(s.getInputStream()));  
PrintWriter out = new PrintWriter(s.getOutputStream(), true);  
  
System.out.print("You: ");  
out.println(key.readLine());  
System.out.println(in.readLine());  
s.close();  
}  
}
```

Commands to Execute:

javac RPCServer.java RPCClient.java

Terminal 1: RPCServer.java

Terminal 2: RPCClient.java

Output (Java):

RPC Server started

You: Hello Server

Server received: Hello Server

Practical No. 4

Remote Method Invocation (RMI)

AIM: To implement a simple remote method invocation between a client and server.

Theory: RMI lets a Java client call a method of an object located on another JVM. The RMI registry is used to find the remote object.

Program:

File name: RMIServer.java

```
import java.rmi.*;

public interface Hello extends Remote {
    String sayHello(String name) throws RemoteException;
}

// HelloImpl.java
import java.rmi.server.*;

public class HelloImpl extends UnicastRemoteObject implements Hello {
    HelloImpl() throws Exception { super(); }

    public String sayHello(String name) {
        return "Hello " + name + " from RMI Server";
    }
}

// Server.java
import java.rmi.*;
import java.rmi.registry.*;

public class Server {
    public static void main(String[] args) throws Exception {
        LocateRegistry.createRegistry(1099);
        Naming.rebind("HelloService", new HelloImpl());
        System.out.println("RMI Server ready");
    }
}
```

File name: RMIClient.java

```
import java.rmi.*;

public class Client {
    public static void main(String[] args) throws Exception {
        Hello h = (Hello) Naming.lookup("rmi://localhost/HelloService");
        System.out.println(h.sayHello("Student"));
    }
}
```

Commands to Execute:

javac RMIServer.java RMIClient.java

Terminal 1: RMIServer.java

Terminal 2: RMIClient.java

Output (Java):

RMI Server ready

Hello Student from RMI Server

Practical No. 5

Increment a Counter in Shared Memory

AIM: To increment a common counter safely using synchronized access.

Theory: Shared memory lets multiple threads/processes access common data. Synchronization prevents two threads from updating the counter at the same time.

Program:

File Name: SharedCounter.java

```
class Counter {
    int value = 0;

    synchronized void increment() {
        value++;
    }
}

public class SharedCounter {
    public static void main(String[] args) throws Exception {
        Counter c = new Counter();

        Runnable job = () -> {
            for (int i = 0; i < 1000; i++)
                c.increment();
        };

        Thread t1 = new Thread(job);
        Thread t2 = new Thread(job);
        t1.start(); t2.start();
        t1.join(); t2.join();

        System.out.println("Final counter = " + c.value);
    }
}
```

Commands to Execute:

```
javac SharedCounter.java  
SharedCounter.java
```

Output:

Final counter = 2000

Practical No. 6

Implement Stateful Server and Stateless Server

AIM: To understand and implement stateful and stateless server behavior.

Theory: A stateful server keeps information about a client between requests. A stateless server treats every request independently and does not keep client session data.

Program:

File name: ServerDemo.java

```
import java.util.*;

public class ServerDemo {
    static Map<String, Integer> state = new HashMap<>();

    static void stateful(String client) {
        int count = state.getDefault(client, 0) + 1;
        state.put(client, count);
        System.out.println("Stateful " + client + ": request " + count);
    }

    static void stateless(String client) {
        System.out.println("Stateless " + client + ": request handled");
    }

    public static void main(String[] args) {
        stateful("C1");
        stateful("C1");
        stateless("C1");
        stateless("C1");
    }
}
```

Commands to Execute:

```
javac ServerDemo.java
ServerDemo.java
```

Output:

Stateful C1: request 1

Stateful C1: request 2

Stateless C1: request handled

Stateless C1: request handled

Practical No. 7

Implement Network File System (NFS)

AIM: To mount and access a shared directory using NFS under Linux.

Theory: NFS allows a client machine to access a directory exported by a server over a network. The client mounts that directory and uses it like a local folder.

Program:

Run on NFS SERVER

```
sudo apt update
sudo apt install nfs-kernel-server -y
sudo mkdir -p /srv/nfs/share
sudo chmod 777 /srv/nfs/share
echo "Distributed file demo" | sudo tee /srv/nfs/share/test.txt
echo "/srv/nfs/share *(rw,sync,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -rav
sudo systemctl restart nfs-kernel-server
```

Run on NFS CLIENT

```
sudo apt install nfs-common -y
sudo mkdir -p /mnt/nfs
sudo mount SERVER_IP:/srv/nfs/share /mnt/nfs
cat /mnt/nfs/test.txt
```

Commands to Execute:

Server: sudo exportfs -rav

Client: sudo mount SERVER_IP:/srv/nfs/share /mnt/nfs

Client: cat /mnt/nfs/test.txt

Output:

Distributed file demo

Practical No. 8

Study RPCgen Protocol Compiler and Find Minimum of Two Numbers

AIM: To use rpcgen under Linux to implement a minimum-number RPC using command-line arguments.

Theory: rpcgen generates the client and server support code from an RPC interface file. The generated program uses RPC stubs so the client can call the remote function.

Program:

RPC File name: min.x

```
program MIN_PROG {  
    version MIN_VERS {  
        int MINIMUM(int, int) = 1;  
    } = 1;  
} = 0x20000001;
```

File name: min_client.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include "min.h"  
  
int main(int argc, char *argv[]) {  
    if (argc != 4) {  
        printf("Usage: %s host a b\n", argv[0]);  
        return 1;  
    }  
  
    CLIENT *clnt = clnt_create(argv[1], MIN_PROG, MIN_VERS, "tcp");  
    if (!clnt) { clnt_pcreateerror(argv[1]); return 1; }  
  
    int a = atoi(argv[2]), b = atoi(argv[3]);  
    int *result = minimum_1(&a, &b, clnt);  
  
    if (result) printf("Minimum = %d\n", *result);  
    else clnt_perror(clnt, "RPC call failed");  
  
    clnt_destroy(clnt);  
    return 0;  
}
```

File name: min_server.c

```
#include "min.h"
```

```
int *minimum_1_svc(int *a, int *b, struct svc_req *req) {  
    static int result;  
    result = (*a < *b) ? *a : *b;  
    return &result;  
}
```

Commands to Execute:

For Generate files: rpcgen -C min.x

For Compile the generated client and server files:

```
gcc -o min_server min_svc.c min_server.c min_xdr.c  
gcc -o min_client min_clnt.c min_client.c min_xdr.c
```

For Start RPC service/port mapper if required:

```
sudo systemctl start rpcbind
```

In Terminal 1

```
./min_server
```

Terminal 2

```
./min_client localhost 15 9
```

Output:

```
Minimum = 9
```

Practical No. 9

Non-token Based Algorithm for Mutual Exclusion

AIM: To implement a simple request-reply non-token based mutual exclusion algorithm.

Theory: In a non-token approach, a process requests permission from other processes before entering the critical section. It enters only after receiving all required replies.

Program:

File name: MutualExclusion.java

```
import java.util.*;

public class MutualExclusion {
    static boolean[] reply = {true, true, true};

    static boolean request(int id) {
        System.out.println("P" + id + " requests critical section");
        for (int i = 0; i < reply.length; i++)
            if (i != id)
                System.out.println("P" + i + " replies OK");
        return true;
    }

    public static void main(String[] args) {
        int p = 1;

        if (request(p)) {
            System.out.println("P" + p + " enters critical section");
            System.out.println("P" + p + " exits critical section");
        }
    }
}
```

Commands to Execute:

```
javac MutualExclusion.java
MutualExclusion.java
```

Output:

P1 requests critical section
P0 replies OK
P2 replies OK
P1 enters critical section
P1 exits critical section

Practical No. 10

Edge Chasing Distributed Deadlock Detection Algorithm

AIM: To simulate edge chasing for detecting a distributed wait-for cycle.

Theory: Edge chasing sends a probe along wait-for edges. If a probe returns to its initiating process, a cycle is found and a distributed deadlock is detected.

Program:

File name: EdgeChasing.java

```
public class EdgeChasing {
    static int[][] wait = {
        {0, 1, 0}, // P0 waits for P1
        {0, 0, 1}, // P1 waits for P2
        {1, 0, 0}  // P2 waits for P0
    };
};

static boolean chase(int start, int current, boolean[] seen) {
    for (int next = 0; next < 3; next++) {
        if (wait[current][next] == 1) {
            System.out.println("Probe: P" + start +
                               " -> P" + current + " -> P" + next);

            if (next == start) return true;

            if (!seen[next]) {
                seen[next] = true;
                if (chase(start, next, seen)) return true;
            }
        }
    }
    return false;
}

public static void main(String[] args) {
    boolean[] seen = new boolean[3];
    seen[0] = true;
```

```
        if (chase(0, 0, seen))  
            System.out.println("Deadlock detected: cycle found");  
        else  
            System.out.println("No deadlock detected");  
    }  
}
```

Commands to Execute:

```
javac EdgeChasing.java  
EdgeChasing.java
```

Output:

```
Probe: P0 -> P0 -> P1  
Probe: P0 -> P1 -> P2  
Probe: P0 -> P2 -> P0  
Deadlock detected: cycle found
```

